**Universidad Autónoma de Occidente**
FACULTAD DE INGENIERÍA Y CIENCIAS BÁSICAS
PROGRAMA ACADÉMICO: INGENIERÍA DE DATOS E INTELIGENCIA ARTIFICIAL
CURSO: ETL (G01)

# Taller-2: Construcción de un Pipeline de Datos Batch Confiable

## 1. Resumen del Taller

| Ítem | Requisito |
|---|---|
| **Escenario** | Integrar datos de Spotify desde CSV y datos de los Premios Grammy desde una base de datos relacional de origen. |
| **Producto principal** | Un pipeline batch confiable y un Data Warehouse dimensional confiable. |
| **Orquestación** | Apache Airflow 3.1.8 utilizando la API TaskFlow. |
| **Validación automatizada** | Great Expectations en las puertas de datos crudos y preparados. |
| **Bases de datos evaluadas** | PostgreSQL o MySQL. Cualquier alternativa requiere aprobación previa del instructor. SQLite no es aceptado. |
| **Analítica** | Al menos tres KPIs y tres visualizaciones consultando el Data Warehouse. |
| **Entrega** | Un repositorio de GitHub organizado y reproducible. |
| **Evaluación** | Solución de Ingeniería / Repositorio: 80%; Presentación y Defensa: 20%. |

**PRINCIPIO DE INGENIERÍA** Un Ingeniero de Datos no construye scripts. Un Ingeniero de Datos diseña sistemas.

## 2. Propósito

Diseñar, implementar, validar, orquestar, monitorear y documentar un pipeline de datos batch confiable que integre dos fuentes de datos heterogéneas y produzca un Data Warehouse analítico confiable.

Este taller evalúa un sistema de ingeniería, no una colección de scripts desconectados. Una solución completa debe conectar los requerimientos analíticos con los datos necesarios, controles de calidad, transformaciones, estructuras dimensionales y salidas analíticas.

**IDEA CLAVE** Los pipelines de datos confiables no asumen que los datos son correctos. Los verifican.

**IMPORTANTE** Confiable no significa que nada falle. Confiable significa que los fallos son anticipados, controlados y observables.

## 3. Resultados de Aprendizaje e Ingeniería

- Definir requerimientos analíticos que exijan información de ambas fuentes de datos.
- Perfilar ambas fuentes e identificar riesgos de calidad de datos basados en evidencia.
- Convertir los hallazgos del perfilado en reglas de calidad explícitas, métricas, umbrales y niveles de severidad.
- Implementar validación automatizada con Great Expectations.
- Diseñar puertas de validación de datos crudos (raw-data) y datos preparados (prepared-data).
- Aplicar reglas justificadas de limpieza, armonización, integración y transformación.
- Diseñar e implementar un Data Warehouse dimensional guiado por requerimientos.
- Construir un DAG de Airflow que orqueste extracción, validación, transformación, validación y carga.
- Configurar un manejo de fallos apropiado y reintentos selectivos.
- Demostrar tanto la ejecución exitosa como el comportamiento ante fallos controlados.
- Interpretar estados de tareas y registros (logs) a través de la UI Web de Airflow.
- Demostrar reejecuciones seguras sin efectos duplicados no deseados.
- Construir KPIs y visualizaciones a partir del Data Warehouse.
- Documentar y defender la solución de ingeniería completa.

## 4. Marco de Ingeniería

### 4.1 Cadena de Razonamiento End-to-End

Toda decisión de implementación importante debe ser trazable a través de esta cadena:

| Etapa | Pregunta de ingeniería |
|---|---|
| **Requisito analítico / de negocio** | ¿Qué decisión o pregunta debe soportar el producto de datos? |
| **Datos requeridos** | ¿Qué atributos y fuentes son necesarios? |
| **Perfilado de datos** | ¿Qué revela la evidencia sobre los datos? |
| **Riesgo de calidad** | ¿Cómo podría la evidencia amenazar el requisito analítico? |
| **Dimensión de calidad** | ¿La preocupación es completitud, validez, unicidad, consistencia, oportunidad u otra dimensión justificada? |
| **Regla de calidad** | ¿Qué debe ser cierto? |
| **Métrica / umbral / severidad** | ¿Cómo se mide el cumplimiento, qué es aceptable y cuán grave es el fallo? |
| **Expectativa de GX (Great Expectations)** | ¿Cómo se automatizará la regla? |
| **Resultado de validación** | ¿Qué evidencia produjo la comprobación? |
| **Acción del pipeline** | ¿Debería el pipeline continuar, advertir, poner en cuarentena, detenerse o seguir otra respuesta documentada? |

### 4.2 Patrón de Pipeline Confiable

| ORIGEN | EXTRACCIÓN | VALIDACIÓN CRUDA | TRANSFORMACIÓN | VALIDACIÓN PREPARADA | CARGA | DESTINO |
|---|---|---|---|---|---|---|
| CSV Spotify + DB Grammy | Adquirir datos de origen | ¿Pueden los datos entrar a transformación? | Limpiar, estandarizar, integrar, derivar | ¿Los datos preparados están listos para cargar? | Escribir estructuras dimensionales | DW Confiable |

**PRINCIPIO DE INGENIERÍA** La validación detecta violaciones, mide la calidad y provee evidencia. La transformación aplica reglas de ingeniería conocidas para limpiar, estandarizar, armonizar, integrar, convertir y derivar.

**IMPORTANTE** Validación y transformación no son la misma actividad. No modifique los datos únicamente para pasar una validación.

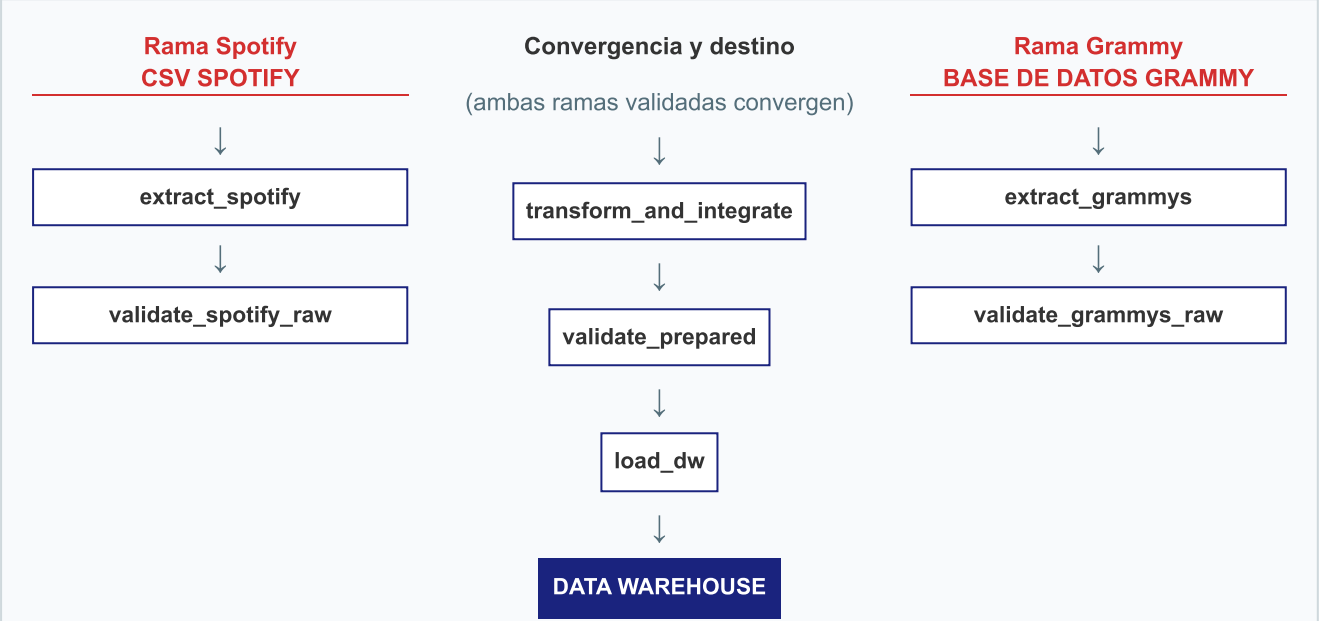
### 4.3 Escenario de Origen

| Origen | Formato provisto | Origen de extracción del pipeline | Aclaración requerida |
|---|---|---|---|
| Spotify | Archivo CSV | Archivo CSV | El pipeline lee los datos de Spotify desde el archivo fuente provisto. |
| Premios Grammy | Archivo CSV | Base de datos origen PostgreSQL o MySQL | Cargar el CSV provisto en esta base de datos es la preparación del origen, no la etapa final de Carga del ETL. |

**EVIDENCIA REQUERIDA** Muestre que el pipeline operativo extrae los datos de Grammy desde la base de datos relacional de origen, no directamente del CSV de Grammy provisto.

## 5. Arquitectura de Solución Requerida

La arquitectura enviada puede usar diferentes nombres de tareas, siempre y cuando preserve ambas ramas de origen, ambas puertas de validación, la dependencia de integración, la carga dimensional y el destino analítico.



**Diseño de calidad rodeando el flujo:** Perfilado → Riesgos de Calidad → Reglas de Calidad → Great Expectations → Puertas de Validación.

**Controles de confiabilidad rodeando el flujo:** Dependencias | Reintentos selectivos | Logs | Monitoreo | Fallo controlado | Reejecución segura.

> **REQUERIDO** El flujo de trabajo completo debe ser orquestado por Apache Airflow 3.1.8. El DAG debe usar la API TaskFlow y la interfaz pública de autoría de `airflow.sdk`. No se requiere `PythonOperator`.

```
from airflow.sdk import dag, task
```

Esta importación es una restricción tecnológica, no una plantilla de solución. El equipo sigue siendo responsable del diseño de tareas, interfaces, dependencias, comportamiento de validación y política de fallos.

# 6. Requerimientos de Trabajo

## 6.1 Requerimientos Analíticos y Alcance de la Solución

Antes del perfilado, validación o modelado dimensional, defina al menos tres requerimientos analíticos que necesiten información de los datos de Spotify y Grammy. Cada requerimiento debe ser lo suficientemente específico para determinar qué datos, transformaciones, controles de calidad, estructuras dimensionales y salidas analíticas son necesarios.

| Requerimiento Analítico | Datos Requeridos / Origen(es) | KPI(s) Esperado(s) | Nivel de Detalle Requerido |
|---|---|---|---|
| Indique la pregunta o decisión que soporta cada requerimiento. | Identifique los atributos necesarios y la fuente de cada uno. | Defina la métrica o KPI esperado. | Explique por qué es necesario combinar ambas fuentes. |

Mantenga la trazabilidad desde cada requerimiento hasta las decisiones de diseño posteriores.

> **EVIDENCIA REQUERIDA** Tabla de requerimientos analíticos completada, una declaración concisa de alcance y referencias de trazabilidad usadas consistentemente en todo el repositorio.

## 6.2 Entorno y Preparación del Origen

| Componente | Elección / restricción requerida |
|---|---|
| **Orquestador** | Apache Airflow 3.1.8 en Docker / Docker Compose. |
| **Autoría de DAG** | Python con la API TaskFlow de Airflow. |
| **Validación** | Great Expectations; comprobaciones automáticas obligatorias. |
| **Base de datos relacional** | PostgreSQL o MySQL. Otras BD requieren aprobación del instructor. SQLite no es aceptado. |
| **Herramienta de BI** | Power BI, Tableau, Looker Studio u otra herramienta aprobada. |
| **Control de versiones** | Git y GitHub. |

Prepare la base de datos de origen de Grammy cargando el CSV provisto en una tabla de origen. Conserve los archivos de entrada originales y documente el proceso de importación, definición de la tabla, conciliación de recuento de filas y cualquier decisión de preparación de origen.

> **IMPORTANTE** Poblar la base de datos de origen de Grammy crea el origen operativo para la extracción. No es la Carga ETL final hacia el Data Warehouse analítico.

- ☐ El entorno Docker Compose inicia de forma reproducible.
- ☐ Airflow reporta la versión 3.1.8.
- ☐ El CSV de Spotify está disponible a través de la ruta documentada del proyecto.
- ☐ La tabla origen de Grammy existe en PostgreSQL/MySQL y concuerda con el CSV provisto.
- ☐ No se suben secretos (commits); `.env.example` documenta las variables requeridas sin credenciales reales.
- ☐ La configuración de conexión y comandos de ejecución están documentados en `README.md`.

## 6.3 Perfilado de Datos y Análisis de Riesgos de Calidad

Perfile ambas fuentes antes de definir comportamientos de limpieza o validación. El perfilado es recolección de evidencia: describe los datos observados y apoya las decisiones de ingeniería. No debe alterar los datos silenciosamente.

| Categoría de Evidencia | Análisis Mínimo |
|---|---|
| **Estructura** | Esquema, columnas, conteos de filas, tipos de datos y campos requeridos por los reqs analíticos. |
| **Completitud** | Conteos de valores faltantes y porcentajes, interpretados en contexto. |
| **Unicidad y duplicación** | Claves candidatas, patrones de duplicados, y si se esperan valores repetidos a nivel de grano de origen. |
| **Contenido categórico** | Valores distintos, patrones de frecuencia, etiquetas inesperadas, variación ortográfica/mayúsculas, y riesgos de dominio. |
| **Contenido numérico** | Rangos, distribuciones, valores imposibles, extremos sospechosos, y unidades donde sea relevante. |
| **Contenido temporal** | Cobertura de fechas, parseabilidad, granularidad, fechas inválidas y brechas relevantes. |
| **Consistencia entre fuentes** | Problemas potenciales de compatibilidad que afecten cruces, comparaciones o integración. |

| Dataset / Atributo | Evidencia del Perfilado | Riesgo Potencial de Calidad | Requerimiento Relacionado |
|---|---|---|---|
| *(Ejemplo)* | *(Ejemplo)* | *(Ejemplo)* | *(Ejemplo)* |

> **EVIDENCIA REQUERIDA** Un notebook de perfilado reproducible más tablas o figuras concisas que respalden cada riesgo reportado. Las capturas de pantalla por sí solas no son evidencia suficiente.

## 6.4 Diseño del Modelo Dimensional

Antes de diseñar la transformación final y lógica de integración, defina el destino analítico que el pipeline debe producir. El modelo dimensional debe derivarse de los requerimientos analíticos y estar soportado por los datos de origen disponibles.

Como mínimo, defina: el proceso de negocio; el grano de la tabla de hechos; las dimensiones; las medidas; claves primarias y foráneas; estrategia de clave subrogada/de negocio; relaciones; cómo se soporta cada requerimiento analítico.

| Decisión de diseño | Contenido requerido |
|---|---|
| **Proceso de negocio** | El proceso representado por la tabla de hechos. |
| **Grano de la tabla de hechos** | Una declaración precisa de lo que representa una fila de hechos. |
| **Dimensiones** | Contextos de análisis descriptivos y sus claves. |
| **Medidas** | Hechos numéricos o contables consistentes con el grano declarado. |
| **Claves y relaciones** | Claves primarias, foráneas, tratamiento de claves subrogadas/de negocio, y restricciones referenciales. |
| **Soporte a requerimientos** | Cómo hechos, dimensiones y consultas responden a los requisitos analíticos definidos. |

## 6.5 Reglas de Calidad de Datos y Diseño de Validación

Convierta los riesgos basados en evidencia en reglas explícitas. Defina al menos seis reglas significativas a través de todo el pipeline. No fuerce un número igual por dataset; la cobertura debe seguir los riesgos reales y requerimientos analíticos.

| ID Regla | Dataset/Capa | Atributo(s) | Dim. de Calidad | Regla de Calidad | Métrica/Umbral | Severidad | Req. Relacionado |
|---|---|---|---|---|---|---|---|
| *(Ej)* | *(Ej)* | *(Ej)* | *(Ej)* | *(Ej)* | *(Ej)* | *(Ej)* | *(Ej)* |

Cada regla debe justificarse por al menos uno de los siguientes: evidencia de perfilado, requerimiento analítico, contrato de origen documentado, o un invariante esperado post-transformación.

| Severidad | Significado | Expectativa de política |
|---|---|---|
| **Critical** (Crítica) | La violación hace que continuar el procesamiento o carga sea inseguro. | Normalmente bloquea la ruta downstream; el equipo debe justificar cualquier excepción. |
| **Warning** (Advertencia) | El problema requiere visibilidad pero puede no invalidar el lote. | Registrar y exponer el resultado; continuar solo bajo una política documentada. |
| **Informational** (Informativa) | El resultado da contexto de monitoreo o señal de tendencia. | Registrar para interpretación; no bloquea independientemente el procesamiento. |

> **IDEA CLAVE** Un umbral es una decisión de ingeniería. Explique por qué es apropiado para el requisito protegido y los datos observados; no lo elija solo porque el lote actual lo pasa.

## 6.6 Diseño de Great Expectations

Implemente las reglas de calidad aprobadas con Great Expectations. Organice el diseño para que el equipo pueda explicar qué objeto de datos se valida, qué Suite de Expectativas (Expectation Suite) aplica, cómo la Definición de Validación (Validation Definition) los conecta, y cómo un Checkpoint o ejecución controlada produce y registra resultados.

| Elemento de diseño | Documentación requerida |
|---|---|
| **Expectativa (Expectation)** | El control GX que implementa una regla de calidad declarada. |
| **Suite de Expectativas** | El grupo coherente de validaciones aplicado a un dataset o capa específica. |
| **Definición de Validación** | La asociación explícita entre la definición de datos y la suite. |
| **Checkpoint/Ejecución** | Cómo se ejecuta la validación, se retienen resultados y se aplica la política. |
| **Resultado de validación** | Éxito general más evidencia útil por regla, no solo captura del valor Booleano. |

**EVIDENCIA REQUERIDA** Un mapeo desde cada Expectativa GX evaluada hacia su ID de Regla, más resultados de validación legibles por máquina (o preservados claramente) para ejecuciones exitosas y fallidas.

## 6.7 Extracción de Datos y Validación Cruda

El DAG final debe contener dos ramas de origen observables de manera independiente:

| Rama Spotify | Rama Grammy |
|---|---|
| `extract_spotify` → `validate_spotify_raw` | `extract_grammys` → `validate_grammys_raw` |

Cada puerta de validación cruda debe responder: *¿Son los datos entrantes lo suficientemente seguros para continuar el procesamiento?*

- Extraiga el origen sin ocultar problemas de calidad mediante limpieza prematura.
- Aplique Expectativas derivadas de las reglas documentadas para ese origen crudo.
- Registre resultados de validación interpretables.
- Detenga el procesamiento subsecuente cuando falle una regla *Critical*.
- Aplique la política documentada a los resultados *Warning* e *Informational*.
- Mantenga salidas o metadatos crudos suficientes para investigar el lote evaluado.

**EVIDENCIA REQUERIDA** Resultados de validación cruda exitosos para ambas fuentes y al menos un fallo de validación cruda Crítica controlado y vinculado a su ID de Regla.

## 6.8 Transformación e Integración de Datos

La transformación es una fase única de ingeniería que puede incluir limpieza, estandarización, armonización, conversión de tipos, manejo de valores nulos, manejo de duplicados, normalización de fechas, derivación e integración cruzada. No cree una etapa de 'limpieza' desconectada solo para reparar fallos de validación.

| Decisión | Explicación requerida |
|---|---|
| **Regla y justificación** | ¿Qué regla de ingeniería conocida se aplica y qué evidencia o requisito la justifica? |
| **Campos afectados / Antes-Después / Manejo de Excepciones** | ¿Qué columnas/registros cambian? ¿Cómo difiere la representación final? ¿Qué pasa cuando un valor no puede transformarse seguramente? |
| **Impacto analítico** | ¿Qué requerimiento, medida, dimensión o paso de integración depende de la decisión? |

**IMPORTANTE** No modifique datos solo para pasar una validación. La transformación debe ser defendible independientemente del resultado actual de validación.

### Contrato de Integración

| Ítem | Decisión del equipo y evidencia |
|---|---|
| **Clave(s) de integración** | Identifique los atributos fuente y la estrategia o clave de coincidencia preparada. |
| **Cardinalidad** | Indique y verifique el comportamiento esperado uno-a-uno, uno-a-muchos, muchos-a-uno, etc. |
| **Preprocesamiento** | Documente la normalización requerida antes del cruce (matching). |
| **Registros no coincidentes** | Defina cómo se miden y tratan las filas que no coinciden. |
| **Coincidencias duplicadas** | Defina cómo se detectan y tratan las coincidencias ambiguas o repetidas. |
| **Supuestos y limitaciones** | Indique qué puede y no puede establecer la integración de forma confiable. |

**IDEA CLAVE** La integración no se documenta adecuadamente con la frase "realizar un merge". Es un contrato de datos con claves, cardinalidad, excepciones, evidencia y consecuencias analíticas.

## 6.9 Validación de Datos Preparados

Después de la transformación e integración, ejecute una segunda puerta de validación obligatoria:

| Entrada | Puerta | Pregunta | Salida |
|---|---|---|---|
| Dataset integrado y preparado | `validate_prepared` | ¿La transformación produjo datos aptos para cargar? | Lote preparado aceptado o fallo controlado |

- Valide el esquema preparado esperado y atributos requeridos.
- Verifique rangos válidos y dominios permitidos que deben mantenerse post-transformación.
- Verifique la integridad de la integración, incluyendo indicadores relevantes de no coincidencia o coincidencia duplicada.
- Rechace valores inválidos críticos que comprometerían la carga dimensional.
- Verifique la preparación analítica para los requerimientos definidos.

| Puerta | Pregunta principal |
|---|---|
| **Validación cruda** | ¿Pueden estos datos entrar seguros a la transformación? |
| **Validación preparada** | ¿La transformación produjo datos aceptables para cargar? |

**EVIDENCIA REQUERIDA** Una Expectation Suite distintiva para datos preparados (o un conjunto de reglas separado), su resultado de ejecución, y prueba de que `load_dw` está bloqueado por su resultado Crítico.

## 6.10 Carga de Data Warehouse y Modelado Dimensional

Implemente el esquema estrella en PostgreSQL o MySQL. El modelo dimensional es obligatorio y debe ser guiado por los requerimientos analíticos.

**IMPORTANTE** La Carga ETL final escribe datos preparados validados en el Data Warehouse dimensional. Es diferente de la importación anterior de preparación de origen de Grammy.

## 6.11 Orquestación de Flujo de Trabajo y Confiabilidad

El DAG de Airflow debe exponer el flujo de trabajo de ingeniería completo en lugar de esconder el pipeline dentro de una tarea. Los límites de las tareas deben soportar estados significativos, logs, aislamiento de fallos y reejecuciones.

| Área | Requisito Mínimo |
|---|---|
| **Tareas** | Representar ambas extracciones, ambas validaciones crudas, transformación/integración, validación preparada y carga a DW. |
| **Dependencias** | Prevenir transformación hasta que ambas ramas crudas cumplan la política; prevenir carga hasta que la validación preparada sea exitosa. |
| **Interfaces de Tareas** | Pasar referencias, rutas, IDs o metadatos compactos; no use el almacén de metadatos del orquestador como transporte de datos masivos. |
| **Condiciones de Fallo** | Hacer explícitos y observables los fallos deterministas de calidad de datos. |
| **Reintentos** | Aplicar selectivamente a fallos operativos transitorios, no indiscriminadamente. |
| **Logging (Registros)** | Registrar contexto de lote, conteos de filas materiales, resultados de validación, resultados de carga y causas de fallo accionables. |
| **Evidencia de Ejecución** | Preservar evidencia exitosa, de fallos controlados y reejecución. |

**PRINCIPIO DE INGENIERÍA** Un DAG es un modelo ejecutable de dependencias y control. Un conjunto de tareas conectadas visualmente no es suficiente a menos que los estados, políticas y evidencias reflejen el comportamiento intencionado del sistema.

### Política de Fallos

| Condición | Severidad / Tipo | Respuesta del Pipeline | ¿Reintento? | Justificación |
|---|---|---|---|---|
| *(Ejemplo)* | *(Ejemplo)* | *(Ejemplo)* | *(Ejemplo)* | *(Ejemplo)* |

| Clase de fallo | Ejemplos | Razonamiento esperado de reintento |
|---|---|---|
| **Operativo transitorio** | Caída temporal de base de datos; interrupción temporal de red. | Un reintento limitado puede ser apropiado porque otro intento puede tener éxito sin cambiar la entrada ni el código. |
| **Determinista de datos/contrato** | Columna faltante; esquema inválido; fallo GX Crítico. | La repetición automática normalmente reproduce el mismo fallo. Detener y mostrar evidencia a menos que exista otra recuperación documentada. |

**IMPORTANTE** Una política generalizada como `retries=3` en todas partes no es evidencia de confiabilidad. Justifique el conteo de reintentos, el retraso y el alcance para cada clase de tarea donde se usen.

### Logging y Observabilidad

Use estados de tareas de Airflow, marcas de tiempo y registros (logs) para responder:

| Pregunta | Evidencia Esperada |
|---|---|
| **¿Qué se ejecutó?** | Identificadores de ejecución de DAG y tarea, contexto del lote/fuente. |
| **¿Cuándo?** | Tiempo lógico/ejecución y marcas de tiempo relevantes de la tarea. |
| **¿Tuvo éxito?** | Estados del DAG y la Instancia de Tarea, más evidencia material de completitud. |
| **¿Dónde falló?** | La tarea fallida y el comportamiento bloqueado/omitido subsecuente. |
| **¿Por qué?** | Mensaje de registro accionable y evidencia de validación u operación vinculada. |
| **¿Qué pasó después?** | Reintento, detención, continuación con advertencia, o comportamiento de recuperación según la política. |

# 7. Pruebas de Confiabilidad Obligatorias

## 7.1 Prueba A - Ejecución Exitosa

Ejecute el pipeline completo con un lote de entrada que satisfaga la política Crítica.

| Etapa | Resultado conceptual esperado |
|---|---|
| **Extraer ambas fuentes** | Éxito |
| **Puertas de validación cruda** | Éxito bajo la política documentada |
| **Transformar e integrar** | Éxito con evidencia de conciliación registrada |
| **Validación preparada** | Éxito |
| **Carga** | Éxito con recuento de filas o evidencia del resumen de carga |
| **Analítica** | Consultas y dashboard reflejan datos del DW |

## 7.2 Prueba B - Fallo de Calidad Crítico Controlado

Introduzca o active intencionalmente una condición que viole una regla de datos crudos Crítica justificada. El equipo elige la condición; este documento no prescribe un valor inválido específico.

| Etapa | Resultado conceptual esperado |
|---|---|
| **Extraer origen afectado** | Éxito |
| **Validación cruda afectada** | Fallo |
| **Transformación downstream** | No procede normalmente |
| **Validación preparada y carga** | No proceden normalmente |
| **Evidencia** | El fallo permanece visible y diagnosticable |

**EVIDENCIA REQUERIDA** Vista Graph o Grid de Airflow, estado de la tarea fallida, resultado de validación GX, log de tarea fallida y una explicación escrita de la regla de bloqueo y el comportamiento downstream.

## 7.3 Repetibilidad y Reejecución Segura

Ejecute el pipeline nuevamente usando el mismo lote de entrada. Demuestre que la reejecución no crea silenciosamente efectos duplicados no deseados en el Data Warehouse.

- Indique la estrategia elegida: reemplazo (replace), truncar-y-cargar, upsert controlado, aplicación de clave de negocio, gestión de clave de lote, u otra aproximación justificada.
- Muestre las claves, restricciones, comandos o lógica de carga relevantes.
- Compare conteos de filas de destino y medidas seleccionadas antes y después de la reejecución.
- Explique qué ocurre tras una ejecución parcialmente completada o fallida.

**PRINCIPIO DE INGENIERÍA** Una reejecución no debe corromper silenciosamente el destino analítico.

### 4.4 Registro de Evidencia de Confiabilidad (Continuación)

| ID Evidencia | Ejecución / Tarea | Artefacto o Ruta | Qué Demuestra | Política / Regla Relacionada |
|---|---|---|---|---|
| *(Ejemplo)* | *(Ejemplo)* | *(Ejemplo)* | *(Ejemplo)* | *(Ejemplo)* |

## 8. Analítica, Trazabilidad y Evidencia

### 8.1 Analítica del Data Warehouse

Construya al menos tres KPIs y/o tres visualizaciones analíticas. Cada salida debe responder o soportar un requerimiento analítico declarado y debe consultar el Data Warehouse dimensional. Conexiones directas a CSV no son aceptadas como solución de dashboard evaluada.

| Requerimiento Analítico | Elemento DW / Consulta | KPI / Visualización |
|---|---|---|
| *(Ejemplo)* | *(Ejemplo)* | *(Ejemplo)* |

**IDEA CLAVE** El dashboard debe responder a los requerimientos analíticos usando datos confiables del Data Warehouse. La complejidad decorativa no es un objetivo de evaluación.

### 8.2 Matriz de Trazabilidad End-to-End

Proporcione suficientes filas representativas para demostrar la cadena de diseño completa. No cree docenas de filas repetitivas; seleccione requisitos y reglas materiales que muestren una trazabilidad end-to-end genuina.

| Req. | Datos Req. | Riesgo Calidad | Regla DQ | Expectativa GX | Transformación | Elemento DW | KPI/Visual. |
|---|---|---|---|---|---|---|---|
| *(Ej)* | *(Ej)* | *(Ej)* | *(Ej)* | *(Ej)* | *(Ej)* | *(Ej)* | *(Ej)* |

### 8.3 Conjunto Mínimo de Evidencias

- ☐ Tabla de reqs analíticos y alcance de la solución.
- ☐ Arquitectura conceptual y estructura del DAG implementado.
- ☐ Notebook de perfilado y evidencia de ambas fuentes.
- ☐ Análisis de riesgos y tabla de reglas de calidad.
- ☐ Diseño GX y salidas de validación para capas crudas y preparadas.
- ☐ Registro de decisiones de transformación e integración.
- ☐ Modelo dimensional y definición de esquema ejecutable.
- ☐ Evidencia de ejecución exitosa del DAG.
- ☐ Evidencia de ejecución fallida controlada del DAG, incluyendo logs de tarea fallida.
- ☐ Evidencia de reintentos selectivos o explicación justificada de tareas sin reintento.
- ☐ Evidencia de reejecución segura.
- ☐ Conexión del dashboard al DW y trazabilidad requisito-a-KPI.

**IMPORTANTE** Las capturas de pantalla deben ser legibles, recortadas a la evidencia relevante y acompañadas de interpretación. Una captura de pantalla sin afirmación ni explicación es evidencia de ingeniería débil.

# 9. Entregables en GitHub

## 9.1 Estructura Sugerida del Repositorio

La siguiente estructura es sugerida, no obligatoria. Una alternativa es aceptable siempre que preserve la separación de responsabilidades y sea reproducible.

```
      workshop-2/ ├── dags/ | └── reliable_music_pipeline.py ├── src/ | ├── extract.py | ├──
transform.py | ├── validation.py | └── load.py ├── gx/ ├── notebooks/ | └──
data_profiling.ipynb ├── sql/ | ├── source_setup.sql | └── dw_schema.sql ├── docs/ | ├──
architecture.* | ├── quality_rules.* | └── evidence/ ├── requirements.txt ├── docker-
compose.yaml ├── .env ├── .gitignore └── README.md
```

**PRINCIPIO DE INGENIERÍA** Separe la lógica de orquestación de la lógica reutilizable de extracción, transformación, validación y carga cuando sea práctico. El DAG debe comunicar la estructura del flujo de trabajo en lugar de volverse un script monolítico de procesamiento.

| Entregable | Contenidos requeridos |
|---|---|
| **1. Perfilado y Diseño de Calidad** | Notebook de perfilado; evidencia; riesgos de calidad; tabla de reglas; enlaces a reqs analíticos. |
| **2. ETL e Integración de Datos** | Lógica de extracción; lógica de transformación e integración; reconciliación; supuestos y limitaciones documentados. |
| **3. Validación Automatizada de Datos** | Configuración GX; Expectativas y Suites; Definiciones de Validación / diseño; resultados crudos y preparados. |
| **4. Pipeline de Airflow Confiable** | DAG; dependencias; reintentos selectivos; manejo de fallos; logs; ejecución exitosa; ejecución fallida controlada; reejecución segura. |
| **5. Producto de Datos Analítico** | DW Dimensional; esquema estrella; consultas KPI; dashboard conectado al DW. |
| **6. Repositorio de Ingeniería** | Repositorio GitHub organizado; configuración reproducible; README; diagramas; documentación; registro de evidencia. |

## 9.2 Requerimientos del README

- Problema y objetivo analítico
- Requerimientos analíticos
- Fuentes de datos
- Arquitectura del pipeline
- Hallazgos del perfilado
- Riesgos y reglas de calidad
- Diseño de validación GX
- Estrategia de transformación e integración
- Modelo dimensional
- Diseño de DAG en Airflow
- Política de fallos y reintentos
- Evidencia de ejecución exitosa
- Evidencia de fallo controlado
- Estrategia de repetibilidad
- Dashboard y salidas analíticas
- Instrucciones de configuración y ejecución
- Supuestos y limitaciones

**REQUERIDO** La falta de una puerta de validación obligatoria, prueba de fallo controlado, demostración de reejecución segura o dashboard conectado al DW representa la falta de una capacidad del sistema, no solo una omisión de documentación.

## 10. Evaluación

### 10.1 Solución de Ingeniería / Repositorio - 4.00 puntos (80%)

| Criterio | Descripción | Puntos |
|---|---|---|
| **Requerimientos Analíticos y Arquitectura** | Requerimientos, datos requeridos, arquitectura y trazabilidad end-to-end. | 0.40 |
| **Perfilado y Riesgos de Calidad** | Perfilado basado en evidencia, riesgos, dimensiones, reglas, métricas y umbrales. | 0.60 |
| **ETL Transformación e Integración** | Extracción correcta más limpieza justificada, armonización, integración y conciliación. | 0.60 |
| **Validación Automatizada con GX** | Validación cruda y preparada, Expectativas significativas, resultados y política operativa. | 0.80 |
| **Orquestación y Confiabilidad en Airflow** | DAG, dependencias, manejo de fallos, reintentos selectivos, logs, fallo controlado y repetibilidad. | 0.90 |
| **DW Dimensional y Analítica** | Esquema estrella guiado por requerimientos, consultas KPI y dashboard desde el DW. | 0.40 |
| **Repositorio y Documentación** | Estructura, reproducibilidad, README, evidencia, higiene de seguridad y calidad del código. | 0.30 |
| | **TOTAL** | **4.00** |

### 10.2 Presentación y Defensa - 1.00 punto (20%)

| Criterio | Puntos |
|---|---|
| Explicación clara de la arquitectura y decisiones de ingeniería | 0.35 |
| Demostración de ejecuciones exitosas y con fallos controlados | 0.30 |
| Comprensión técnica durante las preguntas | 0.25 |
| Comunicación y profesionalismo | 0.10 |
| **TOTAL** | **1.00** |